

# Gold Price Prediction Using LSTM Neural Networks

Javanshir Ismayilzada

April 16, 2025

## Introduction

This project explores the use of Long Short-Term Memory (LSTM) neural networks to predict the daily high price of gold. The model utilizes historical price data, including features such as open, close, volume, and previous high prices, to forecast the next day's highest value. This report presents the methodology, model structure, training process, and evaluation metrics.

## Problem Definition

The goal of this project is to develop a deep learning model capable of predicting the highest price of gold on a given day using the past 60 days of financial indicators. This task is significant in finance, particularly in algorithmic trading and risk management.

# Dataset and Preprocessing

The dataset used contains the following columns:

- **Date:** The date of the record
- **Open:** The opening price of gold
- **High:** The highest price of the day (target)
- **Close:** The closing price of the day
- **Volume:** Trading volume

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load CSV
df = pd.read_csv('GOLD.csv')

# Convert 'Date' column to datetime
df['Date'] = pd.to_datetime(df['Date'])

# Sort by date ascending
df.sort_values('Date', inplace=True)

# Feature columns for model
features = ['Open', 'High', 'Close', 'Volume']
target_column = 'High'

# Normalize the features
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(df[features])
```

Figure 1: \*  
Loading and preprocessing the gold price dataset.

## Sequence Generation for LSTM

We use a window of the past 60 days of data to predict the high price on the next day.

```
SEQ_LEN = 60
X = []
y = []

# Create sequences of 60 days and corresponding target
for i in range(SEQ_LEN, len(scaled_data)):
    X.append(scaled_data[i-SEQ_LEN:i])
    y.append(scaled_data[i, features.index(target_column)])

X, y = np.array(X), np.array(y)

# Train-test split (no shuffle)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False
)
```

Figure 2: \*  
Generating input sequences and target values.

# Model Architecture and Training

The model consists of stacked LSTM layers with dropout and dense layers for final prediction.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dropout, Dense

# LSTM model architecture
model = Sequential()
model.add(LSTM(128, return_sequences=True, input_shape=(X_train.shape[1],
X_train.shape[2])))
model.add(Dropout(0.2))
model.add(LSTM(128))
model.add(Dropout(0.2))
model.add(Dense(64))
model.add(Dense(1))

# Compile and train
model.compile(optimizer='adam', loss='mean_squared_error')
model.fit(X_train, y_train, epochs=40, batch_size=64, validation_data=
(X_test, y_test))
```

Figure 3: \*  
Defining and training the LSTM model.

## Prediction and Evaluation Code

Once the model is trained, we generate predictions and evaluate the performance using common regression metrics.

```
predicted_scaled = model.predict(X_test)

predicted_highs = []
real_highs = []

for i in range(len(predicted_scaled)):
    temp_input = X_test[i, :-1].copy()

    # predicted
    temp_input[features.index(target_column)] = predicted_scaled[i].item()
    predicted_high = scaler.inverse_transform([temp_input])[0]
    [features.index(target_column)]
    predicted_highs.append(predicted_high)

    # actual
    temp_input[features.index(target_column)] = y_test[i].item()
    actual_high = scaler.inverse_transform([temp_input])[0]
    [features.index(target_column)]
    real_highs.append(actual_high)

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

mse = mean_squared_error(real_highs, predicted_highs)
rmse = np.sqrt(mse)
mae = mean_absolute_error(real_highs, predicted_highs)
r2 = r2_score(real_highs, predicted_highs)

print(f"💡 MAE : {mae:.2f}")
print(f"💡 RMSE : {rmse:.2f}")
print(f"💡 R2 : {r2:.4f}")
```

Figure 4: \*  
Making predictions, calculating errors, and evaluation metrics.

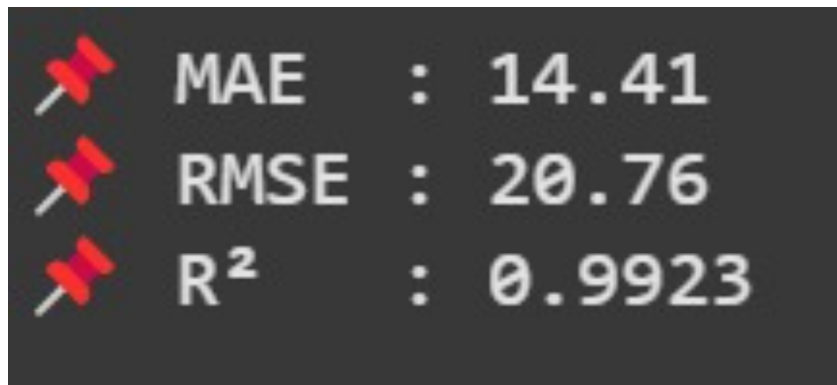


Figure 5: \*  
Summary of Evaluation Metrics — MAE, RMSE, R<sup>2</sup>.

## Evaluation and Results



Figure 6: \*  
Result: Predicted vs Actual High Prices of Gold.

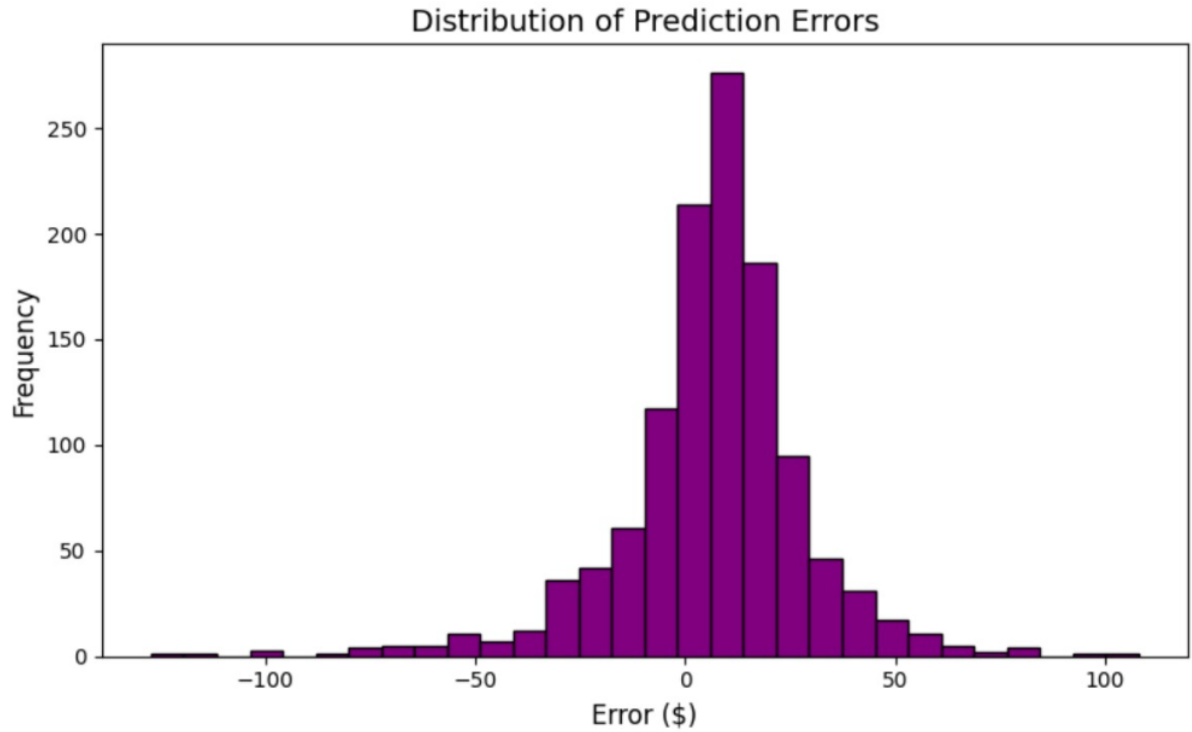


Figure 7: \*  
Result: Histogram of prediction error distribution.

## Conclusion

- The LSTM model performed well, explaining over 99% of the variation in high prices.
- Predictions followed actual trends closely, suggesting practical value for forecasting.
- Future enhancements may include incorporating external factors such as news sentiment, interest rates, and geopolitical events.